

Weather Data Pipeline

Real-time weather collection, processing, storage and visualization with Python

Prepared by: Aakif

Project type: Python data engineering portfolio project

Core technologies: OpenWeatherMap API, Requests, SQLite, Matplotlib and Tkinter

Report date: 6 September 2026

Disclaimer some images and parts in this report was done by the use of AI

1. Executive Summary and Contents

This project is a compact end-to-end weather data pipeline written in Python. It retrieves current observations for selected cities from the OpenWeatherMap API, transforms the response into a smaller and consistent record, stores each observation in SQLite, and presents the history through either a command-line chart or a desktop graphical interface.

The value of the project is not the complexity of the weather data itself. Its value is that it demonstrates the full movement of data through an application: extraction from an external service, validation and conversion, persistent storage, retrieval, visualisation, and user interaction. This makes it useful as a student portfolio project because the design can be explained clearly in an interview.

Reviewed outcome: The supplied project was functional in structure. Four automated tests passed after a security and repository-readiness cleanup. The sample database contains three observations representing three Malaysian cities.

2. Introduction and Objectives

Why the project was developed and what it is intended to demonstrate

Weather information changes continuously, so it is a suitable example for learning how a data pipeline interacts with a live external source. A manual approach would require a person to open a website, copy values and organise them. This application replaces that repeated work with a consistent programmatic process.

Project objectives

- Connect to a public REST API and request weather data for configurable cities.
- Select only the fields needed by the application rather than storing the full response.
- Convert temperatures from Kelvin to Celsius and standardise timestamps in UTC.
- Persist observations in a lightweight relational database for later analysis.
- Provide both a command-line workflow and a user-friendly desktop interface.
- Handle common failures such as invalid keys, unknown cities, timeouts and API rate limits.

Scope

The current scope is a local, single-user application. The default cities are Kuala Lumpur, Klang and Petaling Jaya, although the command line and GUI both allow alternatives. The pipeline collects current conditions rather than forecasts or historical data purchased from a provider. SQLite is used locally, so no database server is required.

Success criteria

The project is successful when it can create the database, request a city, turn the response into a valid record, store that record, reload it and display it without freezing the interface. The program should also fail clearly when a dependency, API key, network connection or expected response field is missing.

3. System Design and Architecture



The project follows a simple extract-transform-load-present design. Each stage has one responsibility, which makes the application easier to test and explain. The command-line script acts as the clearest reference implementation. The GUI contains equivalent pipeline functions and adds threaded execution so the window stays responsive while network requests are running.(image generated through AI)

Component responsibilities

Component	Responsibility
OpenWeatherMap	Returns current weather as JSON for a requested city.
Requests	Sends the HTTP request with query parameters and a timeout.
Transformation logic	Extracts required fields, converts units and formats UTC time.
SQLite	Stores observations in a local relational table.
Matplotlib / Tkinter	Displays trends, cards, status messages and data history.

Design choice: Small functions such as `fetch_weather_data()`, `process_weather_data()` and `store_weather_data()` separate external I/O from deterministic processing. This supports unit testing and makes failures easier to locate.

4. ETL Implementation

Extract

The extraction stage sends a GET request to the Current Weather endpoint. City, API key and the standard-unit setting are supplied as request parameters, while a ten-second timeout prevents the application from waiting indefinitely. HTTP errors are mapped to readable messages for authentication failure (401), missing city (404) and rate limiting (429).

Transform

The transformation stage reads the city name, temperature, humidity, condition description and Unix timestamp from the JSON structure. Temperature is converted using the equation below, rounded to two decimal places, and the description is capitalised for presentation.

$$\text{Temperature (}^{\circ}\text{C)} = \text{Temperature (K)} - 273.15$$

The timestamp is converted into an explicit UTC datetime before being stored as a consistent text value. Missing required fields cause the transformation to return no record, which prevents incomplete observations from entering the database.

Load and present

A parameterised INSERT statement loads the cleaned dictionary into SQLite. Using named placeholders keeps the SQL readable and prevents values from being combined with SQL text. Context managers ensure that connections are closed and successful transactions are committed. The presentation stage then queries stored rows by city and time to build a temperature trend line.

Interview explanation: I selected a modular ETL structure because it mirrors real data engineering systems while remaining understandable in a small Python project. Each stage can be changed independently—for example, SQLite could later be replaced by PostgreSQL without redesigning the API request.

5. Database Design and Sample Data

SQLite was selected because it is included with Python, stores data in one file and does not require a server. This makes the project easy for a recruiter or lecturer to download and run. It is appropriate for a local prototype, while a larger multi-user deployment would require a server-based database.

Table schema

Column	Type	Purpose
id	INTEGER	Auto-incrementing primary key
city	TEXT	Returned location name
temperature	REAL	Processed temperature in Celsius
humidity	INTEGER	Relative humidity percentage
description	TEXT	Readable weather condition
timestamp	TEXT	Observation time in UTC

Reviewed sample database

The supplied database contains three rows collected on 6 May 2026. Each city appears once, so this is enough to prove storage and retrieval but not enough for a meaningful trend analysis.

City	Rows	Temperature	Humidity
Klang	1	25.71 °C	83%
Kuala Lumpur	1	24.86 °C	86%
Petaling Jaya	1	25.19 °C	87%

Data quality note: A future version should add a uniqueness rule or duplicate check based on city and timestamp. Scheduled collection would also create enough observations for useful daily and weekly comparisons.

6. Desktop Interface and Visualisation

The Tkinter interface turns the same pipeline into a more accessible desktop application. The user enters an API key and a comma-separated list of cities, then starts collection with one button. The key field is masked by default, and a show/hide control is available for checking input.

Interface elements

- Weather cards show the latest city, condition, temperature, humidity and UTC time.
- The Data History tab uses a scrollable table to display every stored observation.
- The Temperature Chart tab groups observations by city and draws a separate line for each series.
- Status messages and warning dialogs explain whether collection succeeded or why it failed.
- The Clear All Data action requires confirmation before deleting database rows.

Responsiveness

Network requests are executed in a background thread. This is important because an HTTP request may take several seconds, and running it directly on Tkinter's main event loop would make the window appear frozen. When collection finishes, `self.after()` schedules the visual update back on the GUI thread.

Visualisation choice

A line chart was selected because the main analytical question is how temperature changes over time. The x-axis represents timestamps, the y-axis represents degrees Celsius, and colour separates cities. With only one sample per city the current graph proves the mechanism, but multiple scheduled observations are required before the trend becomes informative.

Usability trade-off: Allowing the key to be entered in the GUI helps beginners run the project, but environment variables remain safer for regular use because credentials are not stored in source files.

7. Testing, Review and Results

Testing focused on the deterministic parts of the pipeline so that verification did not depend on network availability or a third-party API account. The two application files were also compiled to bytecode as a syntax check, and the command-line argument parser was executed with the help option.

Automated checks

Test	Expected result	Status
Kelvin conversion	273.15 K becomes 0.0 °C and 300 K becomes 26.85 °C	Passed
Valid response transform	Required values become one clean dictionary	Passed
Missing response fields	Invalid JSON structure is rejected	Passed
Database initialise/store	Table is created and one complete row is retrieved	Passed
Python compilation	Both application scripts contain valid syntax	Passed
CLI help	Argument parser displays run options correctly	Passed

Result interpretation

All four unit tests passed in the reviewed environment. The database schema was readable and contained three complete observations. A live API request was intentionally not used for verification because the supplied credential had already been exposed in source control and should be revoked before further use.

The tests provide confidence in conversion, validation and local storage, but they do not prove the behaviour of the live API or every GUI interaction. A stronger test suite would mock HTTP responses for each status code and use GUI-level tests for button states, dialogs and chart refresh behaviour.

Review conclusion: The project is suitable for a student portfolio after credential rotation and secure Git-history cleanup. Its strongest points are the complete data lifecycle, understandable modular structure, defensive API handling and usable desktop interface.

8. Security, Limitations and Future Work

Security correction

The original project contained an OpenWeatherMap key directly inside the command-line script and in a tracked environment file. A .gitignore file alone cannot protect a secret that has already been committed. The reviewed version therefore removes the key from current files and reads OWM_API_KEY only from the environment. The old key must still be revoked, and the repository history must be replaced or cleaned before the public main branch is considered safe.

Current limitations

- Collection is started manually, so the database does not yet represent a continuous pipeline.
- The command-line and GUI files repeat some pipeline logic, increasing maintenance effort.
- Only current temperature, humidity and description are stored; wind, pressure and coordinates are ignored.
- SQLite and a desktop GUI are designed for one local user rather than remote access or collaboration.
- The sample has only three rows and cannot support reliable trend conclusions.

Recommended development roadmap

Priority	Improvement	Benefit
1	Rotate key and replace exposed Git history	Removes the immediate security risk
2	Move shared ETL functions into one module	Avoids duplicate logic
3	Add APScheduler or operating-system scheduling	Builds a real time series
4	Mock API responses in automated tests	Verifies error paths reliably
5	Add CSV export or a Streamlit dashboard	Improves analysis and sharing

9. Conclusion, Reflection and References

Conclusion

The Weather Data Pipeline demonstrates a complete and understandable flow from an external API to local storage and visual output. The project uses appropriate tools for its scale: Requests for HTTP communication, SQLite for lightweight persistence, Matplotlib for time-series charts and Tkinter for a desktop interface. Its modular functions make the core process easy to explain, test and extend.

The review improved repository quality without changing the project's main purpose. The hard-coded credential was removed, file naming was corrected, paths were made independent of the terminal's working directory, timezone-aware UTC handling was introduced, dependency and setup documentation was added, and four repeatable unit tests were created.

Personal reflection

The main lesson from this project is that a data pipeline is more than an API request. A reliable pipeline must also validate incoming data, transform it consistently, protect credentials, store records safely, communicate failures and provide evidence that the main functions work. I can use this project in an internship interview to explain both the technical workflow and the engineering decisions behind it.

Key skills demonstrated

- Python programming, modular design and exception handling
- REST API integration and JSON processing
- SQL, relational storage and parameterised queries
- Desktop interface development and background threading
- Data visualisation, testing, documentation and GitHub security

Use of AI guidance

I used ChatGPT as a supporting tool during parts of this project, mainly to obtain guidance, clarify concepts and help troubleshoot implementation issues. I reviewed the suggestions, tested the final program and made sure I could explain the code and design decisions. I remain responsible for the submitted work and its accuracy.

References

OpenWeather (2026). Current Weather Data API. <https://openweathermap.org/api/current>

Python Software Foundation (2026). sqlite3 — DB-API 2.0 interface for SQLite databases. <https://docs.python.org/3/library/sqlite3.html>

Python Software Foundation (2026). tkinter — Python interface to Tcl/Tk. <https://docs.python.org/3/library/tkinter.html>

Matplotlib Development Team (2026). Matplotlib documentation. <https://matplotlib.org/stable/>